CODELINE — GRADUATE TECHNICAL ASSIGNMENT

Clinic Appointment & Patient Records API

System Design Document

Prepared by: **Abdullah Bin Khalid Said Al-Hosni**

Task ID:

`86ey0vuw9`

Track: Java 17 + Spring Boot 3.x + REST API + SQL

Database: PostgreSQL / H2 File-Mode

Document Version 1.0

1. Project Overview

I am building a RESTful backend API for a small outpatient clinic management system using Java 17 and Spring Boot 3.x. The system manages doctors, patients, appointment slots, and visit records. It handles the complete appointment lifecycle — booking, cancellation, rescheduling, and diagnosis recording — while maintaining an immutable audit trail of every appointment event through status-based modelling rather than destructive row deletion.

The business logic enforces real clinical constraints: no slot may be double-booked, a patient may not hold two active appointments with the same doctor on the same day, and diagnoses may only be recorded for appointments whose scheduled time has already passed. Cancellations set status to CANCELLED; rescheduling sets status to RESCHEDULED and creates a new BOOKED appointment row linked via a *rescheduled_to* foreign key. The codebase follows a strict Controller → Service → Repository three-layer architecture backed by a fully normalised SQL relational database.

2. Aim

My aim is to design and deliver a production-quality REST API that demonstrates mastery across five competency areas: system design and requirements analysis, object-oriented domain modelling, relational database schema design, RESTful API design for a real business problem, and evidence-based API testing. I treat **appointment status** (BOOKED, CANCELLED, COMPLETED, RESCHEDULED) as a first-class domain concept, and I model rescheduling as a linked-record operation that preserves the original booking row, satisfying both the doctor-schedule-view and patient-history requirements simultaneously without any data loss.

3. Objectives

3.1 Database Design

I will design a normalised relational database schema reaching Third Normal Form, covering all five domain entities — Doctor, Patient, AppointmentSlot, Appointment, and Visit — with proper primary keys, foreign key constraints, and a self-referencing *rescheduled_to* column on the Appointment table to model the rescheduling chain without row deletion or mutation.

3.2 Business Rule Enforcement

I will implement all business rules exclusively in the service layer: slot double-booking prevention (a BOOKED slot is unavailable until cancelled), duplicate-active-appointment prevention (same patient, same doctor, same day), and time-gated visit recording (slot start time must precede the current timestamp and appointment must not be cancelled).

3.3 RESTful API Design

I will expose a RESTful API surface mapping cleanly to real clinical operations using correct HTTP status codes — 201 for resource creation, 404 for missing entities, 409 for business rule conflicts, and 400 for validation failures — with structured JSON error responses containing human-readable messages rather than raw stack traces.

3.4 Layered Architecture

I will apply a strict three-layer separation: Controller handles HTTP concerns only, Service holds all domain logic and enforces all rules, and Repository handles all data access using Spring Data JPA. Spring dependency injection is used throughout so that each layer is independently replaceable and unit-testable without the full stack.

3.5 Testing and Evidence

I will produce comprehensive test evidence via a Postman collection covering all nine API endpoints, including deliberate rule-violation requests — double-booking, same-day duplicate, future-appointment visit, cancelled-appointment visit — to prove that every enforcement mechanism returns the correct HTTP 409 status with a descriptive error body.

4. User Stories & Requirements

4.1 User Stories

Role	I want to…	So that…
As a Receptionist,	I want to register a new doctor with specialty and working hours,	so that patients can discover and book appointments with the right specialist.
As a Receptionist,	I want to generate 30-minute appointment slots for a doctor on a given date,	so that I have defined, bookable time windows to offer to patients.
As a Receptionist,	I want to view a doctor's full daily schedule (booked and free slots),	so that I can efficiently manage workload and answer patient scheduling queries.
As a Patient,	I want to book an available slot with a specific doctor,	so that I receive medical attention at a time convenient for me.
As a Patient,	I want to cancel an appointment without losing my booking history,	so that the clinic's records remain accurate and my audit trail is preserved.
As a Patient,	I want to reschedule to a different slot with both bookings recorded,	so that the original and new bookings are both traceable for full audit purposes.
As a Doctor,	I want to record a diagnosis and prescription for a completed appointment,	so that the patient's medical record is kept current and permanently retrievable.
As a Receptionist / Patient,	I want to retrieve a patient's full visit history across all doctors,	so that a complete clinical picture including cancelled events is always accessible.

4.2 Functional Requirements

- **FR-01:** The system shall support creation of doctor records with name, medical specialty, and defined working hours (work_start, work_end).
- **FR-02:** The system shall generate 30-minute appointment slots within a doctor's working hours for a specified date, preventing duplicate generation for the same time window.
- **FR-03:** The system shall prevent a slot with an active BOOKED appointment from being booked again until that appointment is cancelled.
- **FR-04:** The system shall prevent a patient from holding two active (non-cancelled) appointments with the same doctor on the same calendar day.
- **FR-05:** The system shall record cancellations and rescheduling as status changes on existing rows — never deletions — and shall store a rescheduled_to foreign key

linking the old appointment to the new one.

- **FR-06:** The system shall permit diagnosis and prescription recording only against appointments whose scheduled slot start time has already passed and whose status is not CANCELLED.
- **FR-07:** The system shall return a patient's complete appointment history including all statuses, diagnoses, prescriptions, and rescheduling links, across all doctors.

4.3 Non-Functional Requirements

- **Validation:** All write endpoints must validate input and return HTTP 400 with a descriptive, structured JSON error body — never a raw Java stack trace.
- **HTTP Status Codes:** The API must use 201 for resource creation, 200 for successful reads, 404 for missing entities, and 409 for business rule conflicts, applied consistently.
- **Layered Architecture:** No business logic may reside in controllers; no raw queries may reside in service classes. Strict Controller → Service → Repository separation is enforced.
- **Persistence:** All data must survive application restarts in a SQL database with foreign key constraints enforced at the database level, not only in application code.
- **Testability:** The service layer must be independently unit-testable via Spring dependency injection without requiring a running HTTP server or live database connection.

5. Proposed Project Structure

I have organised the project as a standard Maven Spring Boot application under the **com.clinic** root package. The layout enforces the three-layer pattern through distinct sub-packages: *controller*, *service*, *repository*, and *model*. Entities and DTOs are further separated within model to prevent JPA implementation details from leaking into the API contract. The exception package centralises all HTTP error handling via a global exception handler.

```
clinic-api/
├── pom.xml
├── README.md
├── src/
    ├── main/
    │   ├── java/com/clinic/
    │   │   ├── ClinicApplication.java
    │   │   ├── controller/
    │   │   │   ├── DoctorController.java
    │   │   │   ├── PatientController.java
    │   │   │   ├── AppointmentController.java
    │   │   │   ├── VisitController.java
    │   │   ├── service/
    │   │   │   ├── DoctorService.java        (interface + impl)
    │   │   │   ├── PatientService.java
    │   │   │   ├── AppointmentService.java   (core business logic)
    │   │   │   ├── SlotService.java
    │   │   │   ├── VisitService.java
    │   │   ├── repository/
    │   │   │   ├── DoctorRepository.java
    │   │   │   ├── PatientRepository.java
    │   │   │   ├── AppointmentSlotRepository.java
    │   │   │   ├── AppointmentRepository.java
    │   │   │   ├── VisitRepository.java
    │   │   ├── model/
    │   │   │   ├── entity/
    │   │   │   │   ├── Doctor.java
    │   │   │   │   ├── Patient.java
    │   │   │   │   ├── AppointmentSlot.java
    │   │   │   │   ├── Appointment.java      (@ManyToOne slot, patient; self-ref)
    │   │   │   │   ├── Visit.java            (@OneToOne appointment)
    │   │   │   ├── dto/
    │   │   │   │   ├── request/  (DoctorRequest, BookingRequest, VisitRequest…)
    │   │   │   │   ├── response/ (DoctorResponse, AppointmentResponse…)
    │   │   │   ├── enums/
    │   │   │   │       ├── AppointmentStatus.java   (BOOKED, CANCELLED, COMPLETED, RESCH
EDULED)
    │   │   ├── exception/
```

```
■   ■       ■■■ GlobalExceptionHandler.java  (@RestControllerAdvice)
■   ■       ■■■ BusinessRuleException.java   (409 Conflict)
■   ■       ■■■ ResourceNotFoundException.java (404 Not Found)
■   ■■■ resources/
■       ■■■ application.yml
■       ■■■ schema.sql                       (DDL — optional for H2)
■■■ test/
    ■■■ java/com/clinic/
        ■■■ service/  (unit tests — mocked repos)
        ■■■ controller/ (integration tests)
```

6. Technologies, Libraries & Design Methods

The following table lists every technology, library, design pattern, and algorithm I plan to use, together with the rationale for each choice.

Category	Technology / Library	Version	Purpose
Language	`Java`	17 LTS	Primary development language — modern features, records, sealed classes
Framework	`Spring Boot`	3.2.x	Application bootstrap, auto-configuration, embedded Tomcat
ORM	`Spring Data JPA`	3.x	Repository abstraction over Hibernate; JPQL and derived queries
ORM Provider	`Hibernate`	6.x	JPA provider; schema generation, lazy loading, cascade management
Database (prod)	`PostgreSQL`	15+	Primary persistent SQL database; FK enforcement, ACID transactions
Database (dev)	`H2 (file mode)`	2.x	Dev/test persistent storage without a separate DB server process
Validation	`Jakarta Validation`	3.x	@NotNull, @NotBlank, @Past on DTOs; HTTP 400 on failure
JSON	`Jackson`	2.x	Automatic Java ↔ JSON serialisation via Spring MVC
Build	`Apache Maven`	3.9+	Dependency management, packaging, and lifecycle management
API Testing	`Postman`	latest	Endpoint testing, collection export, rule-violation evidence
Util	`Lombok`	1.18+	@Data, @Builder, @RequiredArgsConstructor to reduce boilerplate
Pattern	Repository Pattern	—	Abstracts all data access behind interface; service stays DB-agnostic

Pattern	Service Layer Pattern	—	Centralises all business logic; keeps controllers and repos thin
Pattern	DTO Pattern	—	Separates API contract (DTOs) from persistence model (Entities)
Pattern	State Machine (implicit)	—	AppointmentStatus drives allowed transitions; prevents invalid state changes
Algorithm	Slot Generation	—	Iterates [workStart, workEnd) in 30-min steps; idempotent — skips existing slots
Algorithm	Conflict Check	—	Single JPQL EXISTS query per booking; O(1) via indexed FK columns

7. Entity-Relationship Diagram (Chen Notation)

The diagram below uses standard Chen notation. Single rectangles represent strong entities (Doctor, Patient, AppointmentSlot, Appointment); a double rectangle marks the weak entity (Visit, which is existence-dependent on Appointment). Diamonds denote relationships; a double diamond marks the identifying relationship RECORDS between Appointment and Visit. Ellipses are attributes; underlined attribute names denote primary keys. Cardinality labels (1, N) appear adjacent to each relationship line.

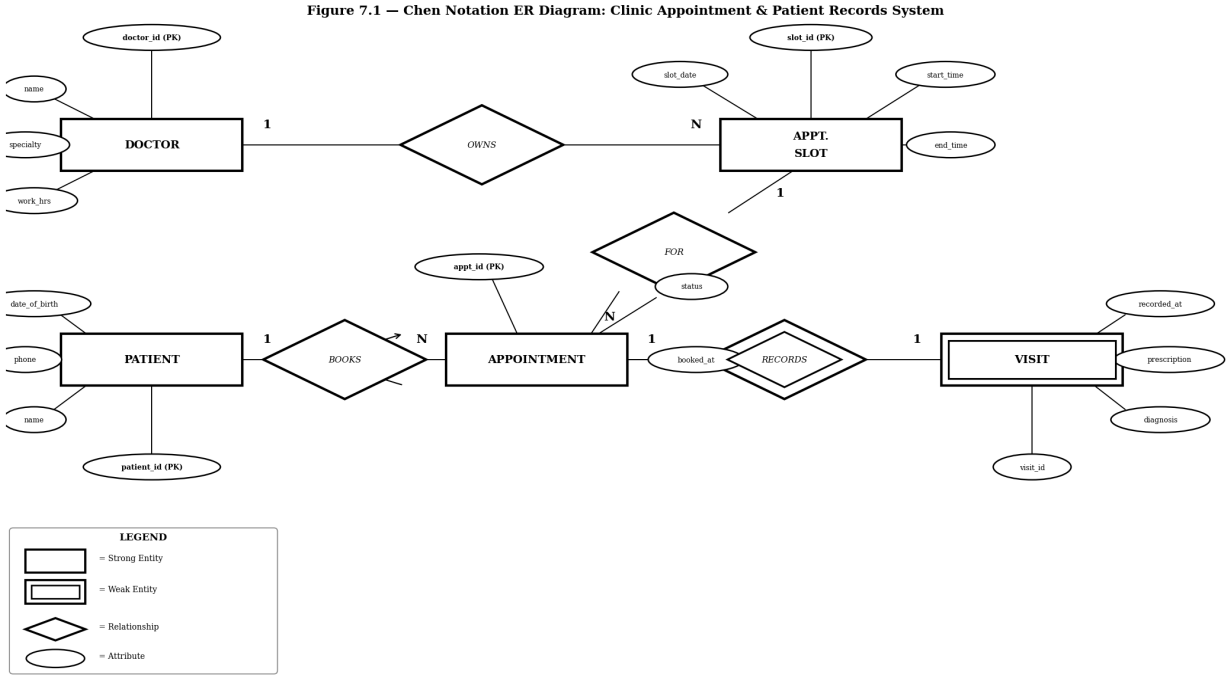

Figure 7.1 — Chen Notation ER Diagram

7.1 Entity Summary

Entity	Type	Primary Key	Key Attributes
Doctor	Strong	`doctor_id`	name, specialty, work_start, work_end
Patient	Strong	`patient_id`	name, date_of_birth, phone
AppointmentSlot	Strong	`slot_id`	doctor_id (FK), slot_date, start_time, end_time
Appointment	Strong	`appointment_id`	patient_id (FK), slot_id (FK), status, booked_at, rescheduled_to (FK self-ref)
Visit	Weak	`visit_id`	appointment_id (FK), diagnosis, prescription, recorded_at

8. Project Pseudo Code

The following pseudo code describes the business logic I will implement for each API endpoint. All rule-enforcement logic lives in the service layer; controllers only translate HTTP requests into service calls and service results into HTTP responses.

8.1 POST /api/doctors — Create Doctor

```
RECEIVE DoctorRequest { name, specialty, workStart, workEnd }
VALIDATE: all fields non-null; workStart < workEnd
  IF invalid → THROW 400 Bad Request with error details
CONSTRUCT Doctor entity from request
SAVE to DoctorRepository
RETURN 201 Created + DoctorResponse { id, name, specialty, workStart, workEnd }
```

8.2 POST /api/patients — Create Patient

```
RECEIVE PatientRequest { name, dateOfBirth, phone }
VALIDATE: all fields non-null; dateOfBirth is in the past; phone non-empty
  IF invalid → THROW 400 Bad Request
CONSTRUCT Patient entity; SAVE; RETURN 201 Created + PatientResponse
```

8.3 POST /api/doctors/{doctorId}/slots — Generate Slots

```
RECEIVE { date }
FIND Doctor by doctorId → IF not found THROW 404
VALIDATE date is not in the past → IF invalid THROW 400
current = doctor.workStart
WHILE current + 30min <= doctor.workEnd:
  IF no AppointmentSlot exists for (doctorId, date, current):
    CREATE AppointmentSlot { doctorId, date, startTime=current, endTime=current+30m }
    SAVE slot
  current = current + 30 minutes
RETURN 201 Created + List<SlotResponse>
```

8.4 GET /api/doctors/{doctorId}/schedule?date= — View Schedule

```
FIND Doctor → IF not found THROW 404
GET all AppointmentSlots for (doctorId, date)
FOR EACH slot:
  booked = AppointmentRepo.existsActiveFor(slot)
  SET slotStatus = booked ? "BOOKED" : "FREE"
RETURN 200 OK + ScheduleResponse { date, slots: [{time, status, patientInfo?}] }
```

8.5 POST /api/appointments — Book Appointment

```
RECEIVE BookingRequest { patientId, slotId }
FIND Patient → IF not found THROW 404
FIND AppointmentSlot → IF not found THROW 404
doctorId = slot.doctor.id; slotDate = slot.slotDate
```

```
-- Rule 1: no double-booking --
IF AppointmentRepo.existsBookedFor(slotId) → THROW 409 "Slot already booked"
-- Rule 2: same patient/doctor/day --
IF AppointmentRepo.existsActiveFor(patientId, doctorId, slotDate)
  → THROW 409 "Patient already has active appointment with this doctor today"
CREATE Appointment { patient, slot, status=BOOKED, bookedAt=NOW }
SAVE; RETURN 201 Created + AppointmentResponse
```

8.6 POST /api/appointments/{id}/cancel — Cancel

```
FIND Appointment by id → IF not found THROW 404
IF appointment.status != BOOKED
  → THROW 409 "Cannot cancel an appointment that is not BOOKED"
appointment.status = CANCELLED    // NO ROW DELETION
SAVE; RETURN 200 OK + AppointmentResponse { status: CANCELLED }
```

8.7 POST /api/appointments/{id}/reschedule — Reschedule

```
RECEIVE { newSlotId }
FIND old Appointment → IF not found THROW 404
IF old.status != BOOKED → THROW 409 "Can only reschedule BOOKED appointments"
FIND newSlot by newSlotId → IF not found THROW 404
IF AppointmentRepo.existsBookedFor(newSlotId)
  → THROW 409 "Target slot is already booked"
-- Create new booking row --
newAppt = CREATE Appointment { patient=old.patient, slot=newSlot, status=BOOKED }
SAVE newAppt
-- Mark old row (NEVER delete) --
old.status = RESCHEDULED
old.rescheduledTo = newAppt
SAVE old
RETURN 200 OK + RescheduleResponse { original, rescheduled }
```

8.8 POST /api/appointments/{id}/visit — Record Visit

```
RECEIVE VisitRequest { diagnosis, prescription }
VALIDATE: both fields non-empty → IF invalid THROW 400
FIND Appointment → IF not found THROW 404
IF appointment.status == CANCELLED
  → THROW 409 "Cannot record visit for a cancelled appointment"
IF appointment.slot.startTime > NOW
  → THROW 409 "Cannot record visit for a future appointment"
IF VisitRepo.existsFor(appointmentId)
  → THROW 409 "Visit already recorded for this appointment"
appointment.status = COMPLETED; SAVE appointment
CREATE Visit { appointment, diagnosis, prescription, recordedAt=NOW }
SAVE visit; RETURN 201 Created + VisitResponse
```

8.9 GET /api/patients/{patientId}/history — Patient History

```
FIND Patient → IF not found THROW 404
```

```
GET all Appointments for patientId (all statuses, ordered by bookedAt DESC)
FOR EACH appointment:
  INCLUDE: slot details (date, time, doctor name)
  INCLUDE: status, bookedAt
  IF appointment has Visit: INCLUDE diagnosis, prescription, recordedAt
  IF appointment.rescheduledTo != null: INCLUDE rescheduled appointment id
RETURN 200 OK + HistoryResponse { patientId, appointments: [...] }
```

9. UML Class Diagram

The UML class diagram below shows the five domain entity classes and the AppointmentStatus enumeration. Each class box is divided into three sections: stereotype (top), class name (middle, bold), attributes (white section), and methods (grey section). Association lines carry cardinality labels; a dashed «uses» line links Appointment to its status enum. The self-referencing arrow on Appointment represents the rescheduledTo relationship used to chain rescheduling events without row deletion.

Figure 9.1 — UML Class Diagram: Clinic Appointment & Patient Records System

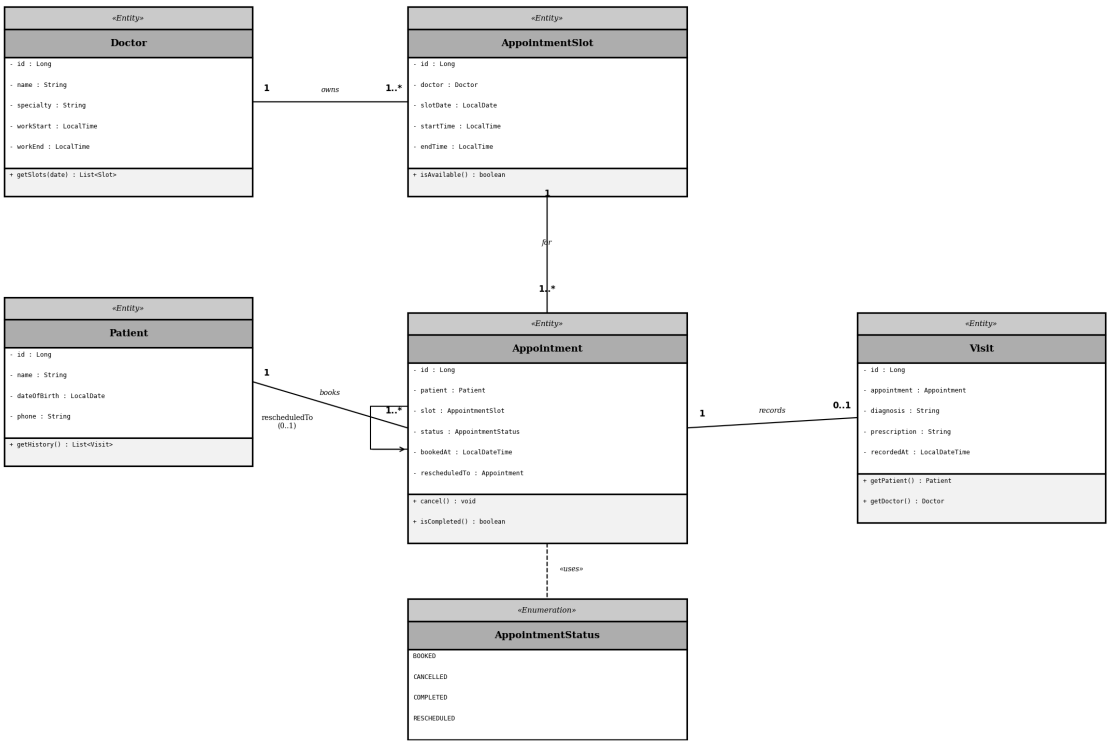

Figure 9.1 — UML Class Diagram

10. Classic ER Diagram (Crow's Foot Notation)

The classic ER diagram below uses Crow's Foot (IE) notation. Entity boxes show the table name in the header and all attributes inside. PK attributes are shown in bold; FK attributes are labelled accordingly. Crow's foot markers on relationship lines indicate cardinality: double perpendicular bars (||) for exactly-one, three-pronged crow's foot for many, and a circle-plus-bar for zero-or-one. The self-referencing relationship on APPOINTMENT represents the rescheduled_to link.

Figure 10.1 — Classic ER Diagram (Crow's Foot): Clinic Appointment & Patient Records System

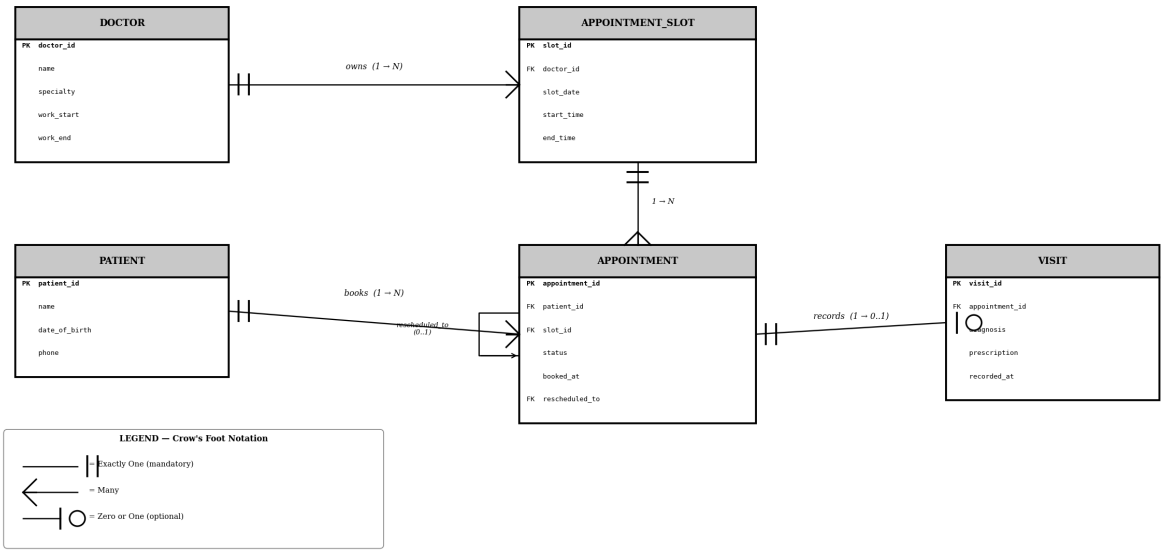

Figure 10.1 — Classic ER Diagram (Crow's Foot)

11. Normalization — Mapping Table (Unnormalized → 1NF → 2NF → 3NF)

I begin with a single flat relation representing all appointment-related data, then progressively decompose it through First, Second, and Third Normal Form to arrive at the five-table schema used in the final implementation.

11.1 Unnormalized / Flat Relation

The unnormalized relation captures every piece of data in one wide row. Patient and doctor details are repeated for every appointment, causing update, insertion, and deletion anomalies.

Column	Anomaly / Issue
`patient_name`	Repeated on every appointment for the same patient
`patient_dob`	Depends only on the patient, not on the appointment
`patient_phone`	Depends only on the patient — update anomaly if phone changes
`doctor_name`	Repeated for every appointment with the same doctor
`doctor_specialty`	Transitive via doctor_name — not directly tied to the appointment
`work_start`	Depends only on the doctor record, not on each appointment
`work_end`	Depends only on the doctor record — denormalized
`slot_date`	Slot facts mixed with appointment facts in same row
`slot_start`	Could be separated into a distinct Slot entity
`slot_end`	Could be separated into a distinct Slot entity
`appt_status`	Single appointment fact — acceptable here as PK candidate
`diagnosis`	Belongs to a Visit concept, not directly to the appointment row
`prescription`	Belongs to a Visit concept — causes NULL columns when no visit
`visit_time`	Belongs to a Visit concept — NULL when appointment not yet completed

11.2 First Normal Form (1NF)

1NF requires: all attribute values are atomic (single-valued), no repeating groups, and every row is uniquely identifiable. I add **appointment_id** as a surrogate primary key. All existing fields are already atomic. The relation is now in 1NF but still contains partial and transitive dependencies.

Attribute	Role	Notes

appointment_id	PK	Surrogate key — uniquely identifies each row
patient_name	Non-key	Atomic — satisfies 1NF but still repeated
patient_dob	Non-key	Atomic and single-valued
patient_phone	Non-key	Atomic
doctor_name	Non-key	Atomic — still denormalized at this stage
doctor_specialty	Non-key	Atomic but transitively dependent on doctor
work_start	Non-key	Atomic but doctor-dependent
work_end	Non-key	Atomic but doctor-dependent
slot_date	Non-key	Atomic
slot_start	Non-key	Atomic
slot_end	Non-key	Atomic
appt_status	Non-key	Atomic — BOOKED/CANCELLED/COMPLETED/RESCHEDULED
diagnosis	Non-key	Atomic but nullable — visit concept mixed in
prescription	Non-key	Atomic but nullable
visit_time	Non-key	Atomic but nullable

Remaining violations: patient_phone depends on patient identity (partial), doctor_specialty and work hours depend on doctor identity (transitive), diagnosis/prescription/visit_time depend on a "visit" sub-concept (transitive).

11.3 Second Normal Form (2NF)

2NF removes partial dependencies — attributes that depend on only part of a composite key. I introduce surrogate IDs and separate patient and doctor details into their own tables, eliminating all repeated data for those entities.

Table	Attributes	Dependency / Action
Doctor	doctor_id (PK), name, specialty, work_start, work_end	Separated — all fields fully depend on doctor_id
Patient	patient_id (PK), name, date_of_birth, phone	Separated — all fields fully depend on patient_id
Appointment_2NF	appointment_id (PK), patient_id (FK), doctor_id (FK), slot_date, slot_start, slot_end, status, diagnosis, prescription, visit_time	Still contains transitive dependencies (slot facts, visit facts)

Remaining violations: In Appointment_2NF, slot_date/slot_start/slot_end describe a reusable Slot concept; diagnosis/prescription/visit_time describe a Visit concept. Both groups are transitively dependent via appointment_id rather than representing appointment-specific facts.

11.4 Third Normal Form (3NF) — Final Schema

3NF eliminates transitive dependencies — every non-key attribute must depend directly on the primary key and nothing else. I extract slot data into AppointmentSlot and visit data into Visit, yielding the clean five-table schema used in the implementation.

Table	Primary Key	Foreign Keys	Remaining Attributes
Doctor	`doctor_id`	—	`name, specialty, work_start, work_end`
Patient	`patient_id`	—	`name, date_of_birth, phone`
AppointmentSlot	`slot_id`	`doctor_id → Doctor`	`slot_date, start_time, end_time`

Appointment	`appointment_id`	`patient_id → Patient`
		`slot_id → AppointmentSlot`
		`rescheduled_to → Appointment`
			`status, booked_at`

Visit	`visit_id`	`appointment_id → Appointment (UNIQUE)`	`diagnosis, prescription, recorded_at`

Verification: In every table, every non-key attribute depends on the whole primary key and nothing but the primary key — meeting the 3NF requirement. The self-referencing FK on Appointment (*rescheduled_to*) enables cancellation without deletion and full rescheduling chain traceability, satisfying Section 3.4 of the assignment without introducing any 3NF violation.